Modelo Predictivo y Asistente Virtual para Cáncer de Mama

1

Caso de estudio: Dataset Breast Cancer Wisconsin (UCI) con integración de API REST y Chatbot

1st PALACIOS MEZA, Briyid Estrella
Facultad de Ingenieria de Sistemas
Universidad Nacional del Centro del Perú.
Huancayo, Perú
e 2025200619H@uncp.edu.pe

2nd HUATUCO BUENDIA, Sebastian Hernan
Facultad de Ingenieria de Sistemas
Universidad Nacional del Centro del Perú.
Huancayo, Perú
e 2025200614c@uncp.edu.pe

3rd BARZOLA CHANCASANAMPA, Marlit Rocio
Facultad de Ingenieria de Sistemas
Universidad Nacional del Centro del Perú
Huancayo, Perú
e 2025100769B@uncp.edu.pe

***Abstract*—Este informe presenta el desarrollo de un sistema de apoyo al diagnóstico médico basado en machine learning. Se utilizó el dataset público Breast Cancer Wisconsin para clasificar tumores como benignos o malignos. El sistema incluye un análisis de datos, entrenamiento de modelos (Regresión Logística y Redes Neuronales), una API para despliegue y un chatbot conversacional. Se priorizó el Recall para minimizar los falsos negativos. Los resultados demuestran que la línea base obtuvo un desempeño sobresaliente en el conjunto de prueba.**

***Index Terms*—Machine Learning, Redes Neuronales, API, Chatbot, Cáncer de Mama.**

I. Planteamiento

El objetivo del proyecto es apoyar al personal de laboratorio clínico en la detección temprana de cáncer de mama. La decisión que apoya el sistema es priorizar la revisión de muestras sospechosas. En el ámbito de la salud, el error más costoso es el \textbf{Falso Negativo} (clasificar como benigno un tumor que es maligno). Por lo tanto, la métrica principal elegida para evaluar el sistema es el \textbf{Recall}. La predicción se realiza en el momento en que el laboratorio obtiene las 30 medidas celulares de la muestra (radio, textura, perímetro, etc.), las cuales están disponibles antes de cualquier decisión clínica.

A. Definición del Error Costoso y la Métrica:

En el ámbito del diagnóstico médico, los errores del modelo tienen implicaciones asimétricas:

- Falso Positivo (FP): Diagnosticar cáncer a una persona sana. Genera ansiedad, costos de exámenes adicionales y estrés innecesario.
- Falso Negativo (FN): Diagnosticar como sano a una persona enferma. Es el error más costoso, ya que retrasa el tratamiento y puede ser letal.

Por lo tanto, la métrica principal de evaluación elegida es el Recall (Sensibilidad), que mide la proporción de casos malignos correctamente identificados. El objetivo es acercar el Recall a 1.0. La predicción se realiza en el momento exacto en que el laboratorio extrae las 30 medidas celulares de la muestra, garantizando que no se utilicen variables desconocidas en el momento de la decisión (evitando la fuga temporal).

II. Datos y Exploración

Se trabajó con el dataset *Breast Cancer Wisconsin (Diagnostic)* obtenido del repositorio UCI Machine Learning. El conjunto contiene 569 instancias y 30 atributos numéricos que describen características de los núcleos celulares (radio, textura, perímetro, área, suavidad, compacidad, etc.). La variable objetivo es binaria: Maligno (M) o Benigno (B).

Calidad y Preprocesamiento:

Se verificó la ausencia de valores nulos. Para robustez, se implementó una imputación por mediana. Para evitar el *Data Leakage* (fuga de información), la división de los datos se realizó antes de cualquier transformación:

1) Entrenamiento (70%): Utilizado para ajustar el modelo y los escaladores.
2) Validación (15%): Utilizado exclusivamente para la selección del mejor modelo e hiperparámetros.
3) Prueba (15%): Reservado para la evaluación final y única del modelo promovido.

La estandarización (media 0, desviación estándar 1) se aplicó ajustando el `StandardScaler` únicamente con el conjunto de entrenamiento (`fit_transform`) y transformando validación y prueba (`transform`), garantizando que el modelo no ”vea” datos futuros.

III. Modelado y Comparación

Se implementaron tres arquitecturas distintas para comparar su desempeño en la tarea de clasificación:

Línea Base (Baseline): Se utilizó una Regresión Logística. Es un modelo lineal simple, altamente interpretable y computacionalmente eficiente. Sirve como punto de referencia para determinar si los modelos más complejos aportan un valor real.

Modelo	Accuracy	Recall	F1-Score
Regresión Logística	0.988	1.000	0.991
Red Neuronal Simple	0.976	0.985	0.980
Red con Dropout	0.988	0.985	0.990

Redes Neuronales Densas (MLP): Se construyeron dos versiones de Perceptrones Multicapa (MLP) utilizando la API Secuencial de Keras/TensorFlow:

- Versión 1 (Simple): Compuesta por una capa de entrada (30 neuronas), una capa oculta de 16 neuronas con activación ReLU, una capa oculta de 8 neuronas y una neurona de salida con activación Sigmoid (para probabilidad binaria).
- Versión 2 (Regularizada): Con la misma arquitectura, pero añadiendo capas de Dropout (0.3 y 0.2). El Dropout apaga aleatoriamente un porcentaje de neuronas durante el entrenamiento, forzando a la red a aprender características más robustas y evitando el sobreajuste (overfitting).

IV. Evaluación y Versionado

Para elegir el modelo ganador, se evaluó el Recall sobre el conjunto de validación. Los resultados fueron los siguientes:

Como se observa, la Regresión Logística obtuvo el mejor Recall, empatando con la red en Accuracy. Esto es común en datasets médicos pequeños (569 filas) donde la relación de las variables es predominantemente lineal. La red con Dropout mostró una ligera mejora frente a la red simple, demostrando que la regularización ayuda a generalizar mejor.

A. Evaluación Final (Test):

El modelo ganador fue evaluado una sola vez en el conjunto de prueba, obteniendo un Recall de 1.0 (detectó todos los casos malignos) y un Accuracy de 98.8%. El modelo fue guardado en formato `.pkl` y su configuración, fecha y métricas se exportaron a un archivo JSON (`ficha_modelo.json`) para garantizar la trazabilidad y reproducibilidad del experimento.

V. Despliegue: API REST con FastAPI

Para poner el modelo en producción, se desarrolló una API web utilizando FastAPI y Uvicorn. La API carga el modelo y el escalador al iniciar el servidor (evitando cargarlo en cada petición para mayor eficiencia).

Endpoints y Validación:

- `GET /salud`: Endpoint de monitoreo que devuelve un estado 200 y confirma que el servicio está operativo.
- `POST /predecir`: Recibe un JSON con las 30 características. Utiliza Pydantic para validar que el usuario envíe exactamente 30 números. Si la validación falla, devuelve un error HTTP 400 con un mensaje comprensible.

Si la entrada es correcta, el modelo procesa los datos, aplica el escalador y devuelve un JSON con la predicción (”Maligno” o ”Benigno”), la probabilidad de benignidad y una advertencia médica obligatoria.

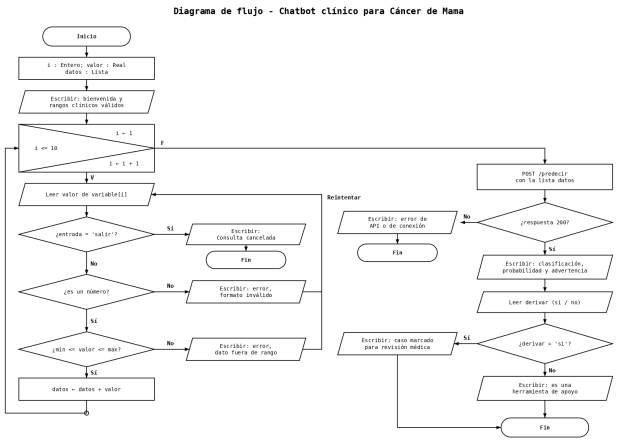


Fig. 1. *Diagrama de flujo del Chatbot*

VI. Asistente Conversacional (Chatbot)

Se desarrolló un Chatbot en Python que actúa como interfaz para el usuario no técnico. El flujo sigue el patrón de recolección de datos por pasos (slots):

1) El bot saluda, muestra los rangos clínicos válidos e inicia la lectura interactiva de las variables de la muestra.
2) En cada paso, valida que el dato sea numérico y se encuentre dentro del rango. Si la entrada es inválida, muestra un mensaje de error (Fallback) y solicita reintentar; además, permite ingresar 'salir' para cancelar la consulta en cualquier momento.
3) Al completar la recolección, consume la API mediante una petición POST /predecir utilizando la librería requests. Si ocurre un fallo de red o error HTTP, notifica el problema de conexión y finaliza.
4) Presenta el resultado en lenguaje natural (clasificación, probabilidad) junto a una advertencia médica obligatoria.
5) Derivación: Si el usuario confirma que desea derivar el caso ('si'), el bot marca la muestra para revisión médica prioritaria; en caso contrario, recuerda que el sistema opera únicamente como una herramienta de apoyo antes de terminar.

VII. Limitaciones y Ética

El sistema es una herramienta de apoyo y nunca debe ser utilizado como un diagnóstico definitivo. Un resultado de ”Maligno” debe ser confirmado por un oncólogo mediante biopsia. Un resultado de ”Benigno” no descarta al 100% la presencia de la enfermedad, por lo que el seguimiento médico es obligatorio.

Desde el punto de vista ético, se decidió no incluir variables sensibles (como edad o raza) para evitar sesgos algorítmicos. Además, el uso de técnicas de regularización y la validación estricta buscan evitar que el modelo memorice datos (overfitting) que podrían fallar en pacientes reales.

VIII. Contribución de cada Integrante

- BARZOLA CHANCASANAMPA, Marlit Rocio: Responsable del análisis exploratorio de los datos (EDA), la limpieza del dataset y la evaluación inicial del modelado. Se encargó de asegurar que los datos estuvieran listos para ser procesados por las redes neuronales.
- PALACIOS MEZA, Briyid Estrella: Encargada de la evaluación y versionado del modelo (selección del modelo ganador y creación de la ficha JSON), así como del desarrollo de la API REST con FastAPI para el despliegue del servicio web.
- HUATUCO BUENDIA, Sebastian Hernan: Responsable del desarrollo del asistente conversacional (Chatbot), la implementación de las pruebas automatizadas con Pytest y la configuración de la integración continua.

IX. Declaración de uso de Inteligencia Artificial

De acuerdo con las reglas de integridad académica de la asignatura, declaramos que se utilizó un asistente de Inteligencia Artificial (Claude) para la estructuración del código, la corrección de errores de entorno (como la instalación de dependencias de TensorFlow en Windows) y la redacción inicial del informe. El diagrama de flujo del chatbot fue diseñado por el equipo basándonos en la lógica real del código. Finalmente, los resultados y métricas presentados en este documento son los arrojados por nuestras propias ejecuciones, no fueron inventados ni alterados por la IA.

References

[1] W. H. Wolberg, W. N. Street, and O. L. Mangasarian, "Breast Cancer Wisconsin (Diagnostic)," UCI Machine Learning Repository, 1995. DOI: 10.24432/C5DW2B.

[2] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[3] M. Abadi et al., "TensorFlow: Large-Scale Machine Learning on Heterogeneous Distributed Systems," *arXiv preprint arXiv:1603.04467*, 2016.

[4] S. Ramírez, "FastAPI: Modern, fast (high-performance), web framework for building APIs with Python," *Tiangolo Documentation*, 2023. [En línea]. Disponible en: https://fastapi.tiangolo.com/